

백트래킹 - 가다가 막히면 되돌아간다

정의

백트래킹은 한 걸음씩 나아가다, 더 이상 답이 될 수 없으면 바로 뒤로 돌아 다른 길을 시도한다. 브루트포스가 모든 길을 끝까지 가 보는 것이라면, 백트래킹은 막힌 길을 일찍 끊는다. 그래서 가지치기(pruning)라고도 한다.

비유: 미로 찾기. 갈림길마다 한 방향으로 들어가 보고, 막다른 길이 나오면 왔던 길로 돌아와 다른 갈림길로 간다.

1. 구조: 재귀 + 되돌리기

핵심은 3단계다.

flowchart TD

```

A[현재 상태] --> B{답이 될 수 있나?}
B -->|아니오| C[가지치기: 이 길은 그만]
B -->|예| D{완성됐나?}
D -->|예| E[기록하고 돌아가기]
D -->|아니오| F[다음 선택을 시도]
F --> G[선택한다 push]
G --> H[재귀 호출]
H --> I[되돌린다 pop]
I --> F
  
```

코드 뼈대:

```

void dfs(int 단계) {
    if (막힌_상태) return;           // 가지치기
    if (완성됐으면) { 기록; return; }

    for (다음_선택 : 후보들) {
        선택.push(다음_선택);          // 가 본다
        dfs(단계 + 1);                // 더 들어간다
    }
}
  
```

```

        선택.pop();                // 되돌아온다 (백트래킹)
    }
}

```

2. 예제: 1~8 중 3개 고르기 (조합)

순서는 상관없고 중복은 안 된다. 예: 1 2 3, 1 2 4, ...

flowchart TD

```

R[시작 start=1] --> A1[1 선택]
A1 --> B1[2 선택]
B1 --> C1[3 선택 → 완성 1 2 3]
B1 --> C2[4 선택 → 완성 1 2 4]
A1 --> B2[3 선택]
B2 --> C3[4 선택 → 1 3 4]
R --> A2[2 선택]
A2 --> B3[3 선택]
B3 --> C4[4 선택 → 2 3 4]

```

start를 쓰는 이유: 이미 고른 수보다 작은 수는 다시 안 본다. 그래야 중복 없이 조합이 된다.

```

#include <bits/stdc++.h>
using namespace std;

int n = 8, k = 3;
vector<int> cur; // 현재 고른 수

void backtrack(int start) {
    if ((int)cur.size() == k) { // 3개를 다 고르면 출력
        for (int x : cur) cout << x << ' ';
        cout << '\n';
        return;
    }
    for (int i = start; i <= n; i++) {
        cur.push_back(i);    // i를 고른다
        backtrack(i + 1);    // 다음은 i보다 큰 수만 고른다
        cur.pop_back();      // 되돌리기 (백트래킹)
    }
}

```

```
int main() {
    backtrack(1);
}
```

출력 일부: 1 2 3 ... 6 7 8 (총 56개)

가지치기로 빨라지는 예: N-Queen

N×N 체스판에 퀸 N개를 서로 공격 못 하게 놓는다.

flowchart TD

```
Q1[1행에 퀸 놓기] --> Q2{같은 열? 같은 대각선?}
Q2 -->|공격 당함| X[가지치기: 이 배치는 버림]
Q2 -->|안전함| Q3[다음 행으로 진행]
Q3 --> Q4{ N개 다 놓았나?}
Q4 -->|예| S[정답 1개 추가]
Q4 -->|아니오| Q1
```

같은 열이나 대각선에 퀸이 있으면 그 갈래는 더 볼 필요 없다. 이 한 줄로 탐색량이 1/10 이하로 준다.

3. 조합 vs 순열: 한 줄 차이

구분	코드 차이	예 (1,2,3 중 2개)
조합 (순서 무시)	<code>backtrack(i+1)</code>	1 2, 1 3, 2 3
순열 (순서 따짐)	방문한 수는 건너뛰고 모두 시도	1 2, 1 3, 2 1, 2 3, 3 1, 3 2

순열 코드 (방문 배열 사용):

```
vector<int> cur;
bool used[9] = {false};

void perm(int depth) {
    if (depth == k) { /* 출력 */ return; }
    for (int i = 1; i <= n; i++) {
        if (used[i]) continue; // 이미 쓴 수는 건너뛰기
        used[i] = true;
```

```

        cur.push_back(i);
        perm(depth + 1);
        cur.pop_back();
        used[i] = false; // 되돌리기
    }
}

```

4. 직접 손으로 풀어 보기

문제 1. 1~4 중 2개를 고르는 조합을 백트래킹 트리로 그려라.

▶ 풀이

문제 2. 순열을 구하는데 `used` 배열을 안 쓰면 무슨 일이 생기나?

▶ 풀이

5. KOI에서는 이렇게 나온다

- "N개 중 K개 고르기", "N-Queen", "스도쿠", "조합의 합"은 백트래킹 대표 유형이다.
- $n \leq 20 \sim 30$ 인데 브루트포스 2^n 이 너무 크면, 백트래킹으로 가지치기가 되는 문제인지 본다.
- 가지치기 조건을 잘못 세우면 답을 놓친다. "이래도 답이 될 수 있는가?"를 정확히 판단해야 한다.
- KOI 중등부는 조합·순열 생성 + 가지치기 한 문제씩은 거의 매년 나온다.